

Report
v. 1.0

Customer
Lombard



Smart Contract Audit

Stake and Bake Native Token

22nd November 2025

Report prepared by
ABDK
Consulting

Contents

1	Changelog	3
2	Summary	4
3	System overview	5
4	Methodology	7
5	Our findings	8
6	Moderate Issues	9
	CVF-1. INFO	9
	CVF-2. INFO	9
	CVF-3. INFO	10
7	Recommendations	11
	CVF-4. INFO	11
	CVF-5. INFO	11
	CVF-6. INFO	11
	CVF-7. FIXED	12
	CVF-8. INFO	12
	CVF-9. FIXED	12
	CVF-10. FIXED	13
	CVF-11. FIXED	13
	CVF-12. INFO	13
	CVF-13. FIXED	13

1 Changelog

#	Date	Author	Description
0.1	22.11.25	A. Zveryanskaya	Initial Draft
0.2	22.11.25	A. Zveryanskaya	Minor revision
1.0	22.11.25	A. Zveryanskaya	Release

2 Summary

All modifications to this document are prohibited. Violators will be prosecuted to the full extent of the U.S. law.

This document presents the results of a smart contract audit performed by ABDK Consulting for **Lombard.Finance** and the **StakeAndBakeNativeToken** System, conducted on 7th November – 15th November 2025 by Mikhail Vladimirov and Dmitry Khovratovich. Our team assessed the security and correctness of the staking and vault deposit workflow, fee accounting, role-based access control, pausing and reentrancy protections, and the integrations with **INativeLBTC** and **IDepositor**. The System is implemented in Solidity 0.8.24 and leverages **OpenZeppelin** upgradeable modules, **SafeERC20**, and **IERC20Permit**, with project-specific components **INativeLBTC**, **IDepositor**, and **Actions**.

Our conclusion is that the System demonstrates solid engineering practices, clear separation of concerns, and appropriate use of **AccessControlUpgradeable**, **ReentrancyGuardUpgradeable**, and **PausableUpgradeable**. The identified findings are primarily Moderate and Minor, with several already remediated. Code quality is good, the upgradability storage pattern is consistent, and the batching logic with per-call gas limits is carefully constrained. Addressing the Moderate items will improve maintainability and gas efficiency.

General recommendations: emit raw revert data in batch flows and decode off-chain, rename **stakeAndBakeInternal** for clarity, trigger **permit** based on payload presence, document fee denominations and maximums near **MAXIMUM_FEE** and initialization parameters, emit change events only when state changes, and remove redundant type conversions. Final verdict: The System is ready for production; ongoing monitoring of access control, pausing, and integration paths is recommended.

It is important to note that a security audit is not a guarantee of absolute security but rather a snapshot of the system's security posture at a specific point in time. While we strive to uncover all potential issues, the evolving nature of blockchain technology and DeFi means new vulnerabilities can emerge.

3 System overview

This section provides a high-level overview of the `StakeAndBakeNativeToken`, a convenience component that mints `LBTC` and stakes the resulting tokens into a vault within a single transaction. The System orchestrates minting, fee handling, permissioned execution, and vault deposit through a controlled workflow so that an operator can atomically deliver vault shares to a designated recipient while applying configurable limits and safeguards.

We were asked to review:

- [Original Code](#)
- [Code with Fixes](#)

Files:

```
/
StakeAndBakeNativeToken.sol
```

The technology stack is `Solidity` 0.8.24, with upgradeable patterns via `AccessControlUpgradeable`, `ReentrancyGuardUpgradeable`, and `PausableUpgradeable`. Token handling relies on `SafeERC20`, `IERC20`, and `IERC20Permit`. The System follows an explicit storage layout convention using a namespaced slot consistent with `ERC-7201` style custom storage. The primary code artifact is `StakeAndBakeNativeToken.sol`. Project-specific interfaces include `INativeLBTC` and `IDepositor`, and a library import `Actions`.

At a conceptual level, core components include a minting adapter, a vault depositor, and an operator-controlled router that binds both actions into one flow. The minting adapter abstracts communication with the `LBTC` issuance mechanism and accepts a payload plus a proof for authorization. The depositor abstracts integration with downstream vaults that may require the caller to hold tokens at the time of deposit, with the expectation that vault shares are delivered to the intended owner. The router encapsulates permissions, pausing, gas budgeting per operation, and fee collection, creating predictable and repeatable execution for end users while keeping mint and stake closely coupled.

The System's roles and operational controls are implemented through `AccessControlUpgradeable`. Administrative configuration is separated from operator permissions using `DEFAULT_ADMIN_ROLE`, `FEE_OPERATOR_ROLE`, `PAUSER_ROLE`, and `CLAIMER_ROLE`. The pausing mechanism allows privileged actors to halt mint-and-stake operations. Reentrancy protection guards external-entry functions, and a self-call pattern is used to enforce that certain internalized steps can only be reached through the

System's own entry points. A per-call gas ceiling is configurable to bound the execution cost of each bundled stake and bake step, including in batch mode.

Token flows center on the native **LBTC** representation. After a successful mint through **INativeLBTC**, the System determines the recipient address from the mint payload and moves the minted amount into the System for staking. The design supports permit-based authorization using **IERC20Permit**, allowing allowances to be set via signatures, and it falls back to standard allowances if already granted. A configurable fee is subtracted from the amount to be staked and sent to a treasury address obtained from the adapter, after which the remaining tokens are deposited into the target vault on behalf of the owner. The fee constant **MAXIMUM_FEE** caps fee configuration.

Vault interactions are abstracted through **IDepositor**. The System increases allowance to the depositor for the staking portion, then invokes the deposit routine with owner context and an opaque payload that the depositor understands. This model isolates vault-specific logic behind a single interface so that the System does not couple to individual vault implementations. The depositor is responsible for returning vault shares to the intended owner. This indirection enables a clean separation between token lifecycle operations and vault-specific mechanics.

Failure handling in batch processing prefers resilience and observability. When batching, the System attempts each unit with a bounded gas limit and records failures through structured events that capture either human-readable messages or low-level data. This approach allows off-chain analysis of partial failures without halting the entire batch. Emphasis is placed on deterministic behavior, explicit revert reasons where practical, and a clear audit trail through events for operational monitoring.

External connections include the **INativeLBTC** adapter for mint authorization and treasury routing, the **IDepositor** integration for vault deposits, and standard token interfaces **IERC20** and **IERC20Permit** for balance movement and permit-based approvals. The System depends on **OpenZeppelin** upgradeable building blocks for permissions, pausing, and reentrancy safety. These external components are all referenced via interfaces to ensure clear boundaries and facilitate upgrades.

Initialization configures the set of roles, binds the token and adapter, sets fee and per-call gas limits, and authorizes the System itself where required for controlled internal dispatch. Administrative actions exist to set the depositor, update the fee within a strict cap, and adjust the gas limit used for each operation. Pausing and unpausing are controlled through designated roles. Read-only accessors expose current parameters for transparency. The constructor disables initializers per upgradeable best practice, and storage is namespaced through a dedicated slot to avoid collisions in proxy deployments.

Overall, the **StakeAndBakeNativeToken** System presents a focused orchestration layer that binds minting, fee handling, and vault deposit into one permissioned workflow. Role-based separation, pausing controls, and bounded gas per operation improve operational safety. Interface-driven dependencies keep the System decoupled from specific vaults or issuance internals, while standard token interfaces and permit support enable flexible user authorization paths. The design is aligned with upgradeable deployment practices and provides a clear event surface for observability during both single and batch executions.

4 Methodology

The methodology is not a strict formal procedure, but rather a selection of methods and tactics combined differently and tuned for each particular project, depending on the project structure and technologies used, as well as on client expectations from the audit.

- **General Code Assessment.** The code is reviewed for clarity, consistency, style, and for whether it follows best code practices applicable to the particular programming language used. We check indentation, naming convention, commented code blocks, code duplication, confusing names, confusing, irrelevant, or missing comments etc. At this phase we also understand overall code structure.
- **Entity Usage Analysis.** Usages of various entities defined in the code are analysed. This includes both: internal usages from other parts of the code as well as potential external usages. We check that entities are defined in proper places as well as their visibility scopes and access levels are relevant. At this phase, we understand overall system architecture and how different parts of the code are related to each other.
- **Access Control Analysis.** For those entities, that could be accessed externally, access control measures are analysed. We check that access control is relevant and done properly. At this phase, we understand user roles and permissions, as well as what assets the system ought to protect.
- **Code Logic Analysis.** The code logic of particular functions is analysed for correctness and efficiency. We check whether the code actually does what it is supposed to do, whether the algorithms are optimal and correct, and whether proper data types are used. We also make sure that external libraries used in the code are up to date and relevant to the tasks they solve in the code. At this phase we also understand data structures used and the purposes they are used for.

We classify issues by the following severity levels:

- **Critical issue** directly affects the smart contract functionality and may cause a significant loss.
- **Major issue** is either a solid performance problem or a sign of misuse: a slight code modification or environment change may lead to loss of funds or data. Sometimes it is an abuse of unclear code behaviour which should be double checked.
- **Moderate issue** is not an immediate problem, but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain business flows.
- **Recommendations** cover code style, best practices and general improvements.

5 Our findings

We provided the client with several recommendations.

Moderate	Info 3	Fixed 0
Minor	Info 5	Fixed 5

6 Moderate Issues

CVF-1 INFO

- **Category** Suboptimal

- **Source**

StakeAndBakeNativeToken.sol

Description Parsing the error message on-chain looks like an overkill.

Recommendation Just emit an event with row data and parse off-chain if necessary.

Client Comment *We implemented this approach in 'StakeAndBake.sol' contract and prefer to keep error handling to be similar in both versions.*

```
195 } catch Error(string memory message) {  
    emit BatchStakeAndBakeReverted(i, message, "");  
} catch (bytes memory lowLevelData) {  
    emit BatchStakeAndBakeReverted(i, "", lowLevelData);  
}
```

CVF-2 INFO

- **Category** Bad naming

- **Source**

StakeAndBakeNativeToken.sol

Description Using the "Internal" suffix for the name of an external function is confusing.

Recommendation Rename the function.

Client Comment *The name is correct since this function includes the logic verifying that it is called by this contract only. We prefer to keep this name the way it is now.*

```
209 function stakeAndBakeInternal(  

```

```
211 ) external returns (bytes memory) {
```

CVF-3 INFO

- **Category** Unclear behavior

- **Source**
StakeAndBakeNativeToken.sol

Description Checking the allowance on-chain to tell whether the permit is required is waste of gas. Let the caller decide.

Recommendation Execute the permit call when data.permitPayload isn't empty, regardless of the current allowance.

Client Comment *The caller of this function is not the owner of token to be minted so if caller would check allowance before calling smart-contract, it is still possible that token owner manages to revoke allowance after the check and before transaction is being set to this smart contract. And transaction will fail in such case.*

```
299 if (
300     IERC20(address($.nativeLbtc)).allowance(owner, address(this)) <
        data.amount
    ) {
        if (data.permitPayload.length < 83) {
            revert InvalidPermitPayload();
        }
    }
```



7 Recommendations

CVF-4 INFO

- **Category** Procedural

- **Source**

StakeAndBakeNativeToken.sol

Description We didn't review these files.

Client Comment *Aknowledged.*

```
4 import {INativeLBTC} from "../LBTC/interfaces/INativeLBTC.sol";  
12 import {Actions} from "../libs/Actions.sol";
```

CVF-5 INFO

- **Category** Suboptimal

- **Source**

StakeAndBakeNativeToken.sol

Description These errors could be made more useful by adding certain parameters into them.

Client Comment *We implemented this approach in 'StakeAndBake.sol' contract and prefer to keep error handling to be similar in both versions.*

```
38 error WrongAmount();  
40 error InvalidPermitPayload();
```

CVF-6 INFO

- **Category** Bad datatype

- **Source**

StakeAndBakeNativeToken.sol

Description The parameter type should be "IDepositor".

Client Comment *We implemented this approach in 'StakeAndBake.sol' contract and prefer to keep error handling to be similar in both versions.*

```
42 event DepositorSet(address indexed depositor);
```

CVF-7 FIXED

- **Category** Documentation

- **Source**

StakeAndBakeNativeToken.sol

Description The number format for this field is unclear.

Recommendation Explain in a documentation comment.

```
69 uint256 fee;
```

CVF-8 INFO

- **Category** Suboptimal

- **Source**

StakeAndBakeNativeToken.sol

Description Solidity compiler is smart enough to precompute constant hash expressions.

Recommendation Use a hash expression instead of a hardcoded hash value.

Client Comment *We are following such approach in all our smart contracts and prefer to have it this way here too.*

```
77 // keccak256(abi.encode(uint256(keccak256("lombardfinance.storage.  
    ↳ StakeAndBakeNativeToken"))) - 1)) & ~bytes32(uint256(0xff))
```

```
79 0x0e47f6bdb2c8c295db7a485b9611b576c67616f4bb0fcb676c069d08170f1800  
    ↳ ;
```

CVF-9 FIXED

- **Category** Documentation

- **Source**

StakeAndBakeNativeToken.sol

Description The number format for this constant is unclear.

Recommendation Explain in a documentation comment.

```
81 uint256 public constant MAXIMUM_FEE = 100000;
```

CVF-10 FIXED

- **Category** Documentation

- **Source**

StakeAndBakeNativeToken.sol

Description The number format for this argument is unclear.

Recommendation Explain in a documentation comment.

```
104 uint256 fee_,
```

CVF-11 FIXED

- **Category** Suboptimal

- **Source**

StakeAndBakeNativeToken.sol

Description This check should be performed earlier.

```
127 if (address(adapter_) == address(0)) {  
    revert ZeroAddress();
```

CVF-12 INFO

- **Category** Suboptimal

- **Source**

StakeAndBakeNativeToken.sol

Description These events are emitted even if nothing actually changed.

Client Comment *We do not see any problem in such approach.*

```
144 emit FeeChanged(fee);
```

```
157 emit GasLimitChanged(gasLimit);
```

```
171 emit DepositorSet(depositor);
```

CVF-13 FIXED

- **Category** Suboptimal

- **Source**

StakeAndBakeNativeToken.sol

Description Double type conversion is redundant, as the "nativeLbts" field is already "IERC20".

```
274 IERC20(address($.nativeLbtc)).safeIncreaseAllowance(
```



ABDK

Consulting

About us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and co-authored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, JavaScript, and other languages.

Contact

Email

dmitry@abdkconsulting.com

Website

abdk.consulting

Twitter

twitter.com/ABDKconsulting

LinkedIn

linkedin.com/company/abdk-consulting